

Algorithms 2

Clustering Algorithms

Daniel Rosenberg & Michael Trushkin

Ariel University

1. The k -Centre Problem

- $[n]$ — n **data points**
- $D := (d_{ij})$ — symmetric distance matrix:
 - $d_{ii} = 0$ for all i
 - $d_{ij} \geq 0$ for all $i, j \in [n]$
- D satisfies the **$\triangle$ -inequality**:

$$\forall i, j, k \in [n] : d_{ij} + d_{jk} \geq d_{ik}$$

- $[n]$ — n **data points**
- $D := (d_{ij})$ — symmetric distance matrix:
 - $d_{ii} = 0$ for all i
 - $d_{ij} \geq 0$ for all $i, j \in [n]$
- D satisfies the **$\triangle$ -inequality**:

$$\forall i, j, k \in [n] : d_{ij} + d_{jk} \geq d_{ik}$$

We view D as a **positively edge-weighted complete graph** on n vertices.

- $[n]$ — n **data points**
- $D := (d_{ij})$ — symmetric distance matrix:
 - $d_{ii} = 0$ for all i
 - $d_{ij} \geq 0$ for all $i, j \in [n]$
- D satisfies the **$\triangle$ -inequality**:

$$\forall i, j, k \in [n] : d_{ij} + d_{jk} \geq d_{ik}$$

We view D as a **positively edge-weighted complete graph** on n vertices.

For $S \subseteq [n]$ and $i \in [n]$, define the **distance from i to S** :

$$d(i, S) := \min_{j \in S} d_{ij}$$

Note: if $i \in S$, then $d(i, S) = 0$.

- $[n]$ — n **data points**
- $D := (d_{ij})$ — symmetric distance matrix:
 - $d_{ii} = 0$ for all i
 - $d_{ij} \geq 0$ for all $i, j \in [n]$
- D satisfies the **$\triangle$ -inequality**:

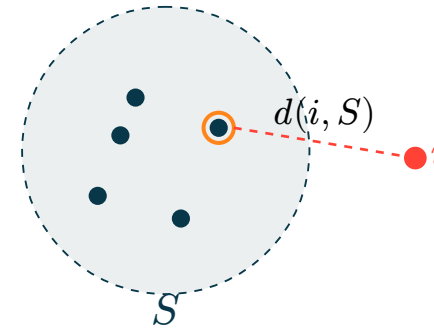
$$\forall i, j, k \in [n] : d_{ij} + d_{jk} \geq d_{ik}$$

We view D as a **positively edge-weighted complete graph** on n vertices.

For $S \subseteq [n]$ and $i \in [n]$, define the **distance from i to S** :

$$d(i, S) := \min_{j \in S} d_{ij}$$

Note: if $i \in S$, then $d(i, S) = 0$.



Problem 1 The k -Centre Problem

Given: $k \in \mathbb{N}$, points $[n]$, distance matrix D satisfying $\triangle$ -ineq.

Find: $S \subseteq [n]$, $|S| = k$, minimising

$$\max_{i \in [n]} d(i, S)$$

Problem 2 The k -Centre Problem

Given: $k \in \mathbb{N}$, points $[n]$, distance matrix D satisfying $\triangle$ -ineq.

Find: $S \subseteq [n]$, $|S| = k$, minimising

$$\max_{i \in [n]} d(i, S)$$

- Members of S are **centres** (cluster centroids)
- $\max_{i \in [n]} d(i, S)$ is the **radius** of S
- Goal: minimise radius — every point close to some centre

Problem 3 The k -Centre Problem

Given: $k \in \mathbb{N}$, points $[n]$, distance matrix D satisfying $\triangle$ -ineq.

Find: $S \subseteq [n]$, $|S| = k$, minimising

$$\max_{i \in [n]} d(i, S)$$

- Members of S are **centres** (cluster centroids)
- $\max_{i \in [n]} d(i, S)$ is the **radius** of S
- Goal: minimise radius — every point close to some centre

Remark

The k -centre problem is **NP-hard**.

Problem 4 The k -Centre Problem

Given: $k \in \mathbb{N}$, points $[n]$, distance matrix D satisfying $\triangle$ -ineq.

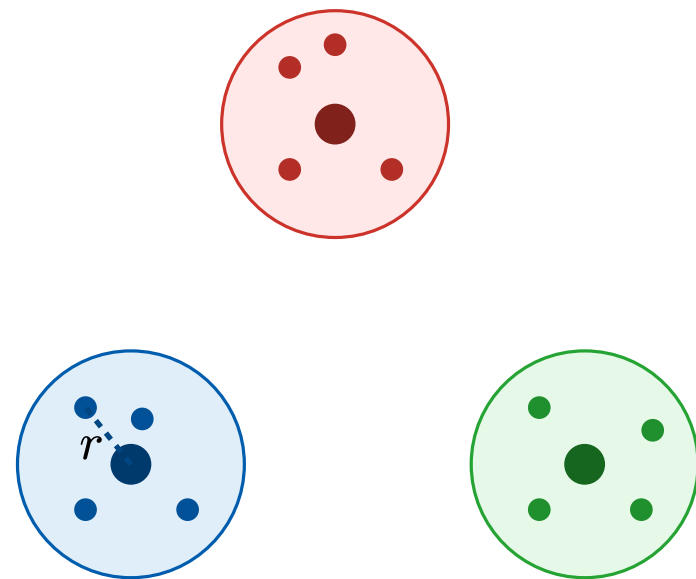
Find: $S \subseteq [n]$, $|S| = k$, minimising

$$\max_{i \in [n]} d(i, S)$$

- Members of S are **centres** (cluster centroids)
- $\max_{i \in [n]} d(i, S)$ is the **radius** of S
- Goal: minimise radius — every point close to some centre

Remark

The k -centre problem is **NP-hard**.



Service points = filled dots

Algorithm 1: Greedy k -Centre

```
1: Input: points  $[n]$ , distances  $D$ , integer  $k$ 
2:  $S \leftarrow \{i\}$  for arbitrary  $i \in [n]$ 
3: while  $|S| < k$  do
4:    $j \leftarrow \arg \max_{j \in [n]} d(j, S)$ 
5:   (pick farthest point from  $S$ )
6:    $S \leftarrow S \cup \{j\}$ 
7: end
8: return  $S$ 
```

Algorithm 2: Greedy k -Centre

```
1: Input: points  $[n]$ , distances  $D$ , integer  $k$ 
2:  $S \leftarrow \{i\}$  for arbitrary  $i \in [n]$ 
3: while  $|S| < k$  do
4:    $j \leftarrow \arg \max_{j \in [n]} d(j, S)$ 
5:   (pick farthest point from  $S$ )
6:    $S \leftarrow S \cup \{j\}$ 
7: end
8: return  $S$ 
```

 Remark

More concisely: start with any point; repeatedly add the point **farthest** from the current set.

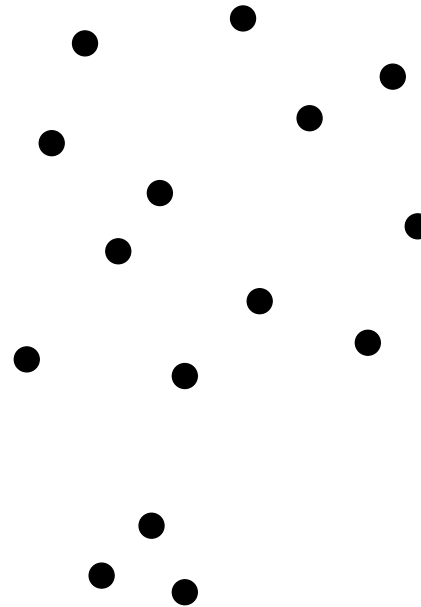
Algorithm 3: Greedy k -Centre

```
1: Input: points  $[n]$ , distances  $D$ , integer  $k$ 
2:  $S \leftarrow \{i\}$  for arbitrary  $i \in [n]$ 
3: while  $|S| < k$  do
4:    $j \leftarrow \arg \max_{j \in [n]} d(j, S)$ 
5:   (pick farthest point from  $S$ )
6:    $S \leftarrow S \cup \{j\}$ 
7: end
8: return  $S$ 
```

Remark

More concisely: start with any point; repeatedly add the point **farthest** from the current set.

Execution example ($k = 4$):



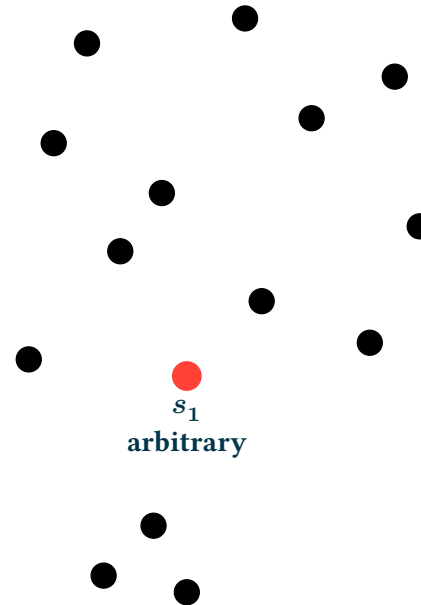
Algorithm 4: Greedy k -Centre

```
1: Input: points  $[n]$ , distances  $D$ , integer  $k$ 
2:  $S \leftarrow \{i\}$  for arbitrary  $i \in [n]$ 
3: while  $|S| < k$  do
4:    $j \leftarrow \arg \max_{j \in [n]} d(j, S)$ 
5:   (pick farthest point from  $S$ )
6:    $S \leftarrow S \cup \{j\}$ 
7: end
8: return  $S$ 
```

Remark

More concisely: start with any point; repeatedly add the point **farthest** from the current set.

Execution example ($k = 4$):



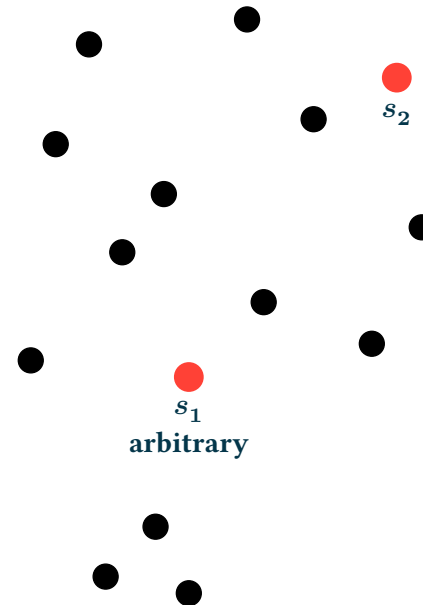
Algorithm 5: Greedy k -Centre

```
1: Input: points  $[n]$ , distances  $D$ , integer  $k$ 
2:  $S \leftarrow \{i\}$  for arbitrary  $i \in [n]$ 
3: while  $|S| < k$  do
4:    $j \leftarrow \arg \max_{j \in [n]} d(j, S)$ 
5:   (pick farthest point from  $S$ )
6:    $S \leftarrow S \cup \{j\}$ 
7: end
8: return  $S$ 
```

Remark

More concisely: start with any point; repeatedly add the point **farthest** from the current set.

Execution example ($k = 4$):



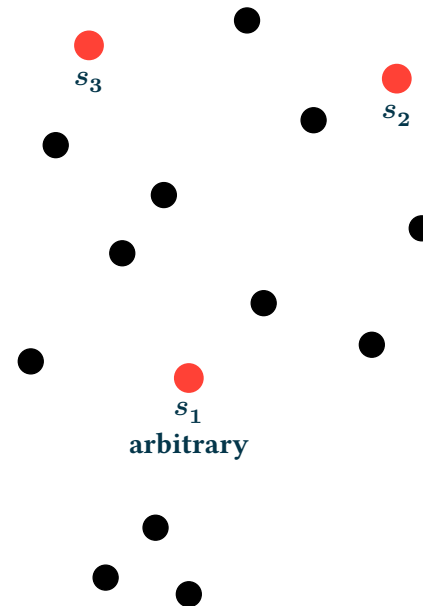
Algorithm 6: Greedy k -Centre

```
1: Input: points  $[n]$ , distances  $D$ , integer  $k$ 
2:  $S \leftarrow \{i\}$  for arbitrary  $i \in [n]$ 
3: while  $|S| < k$  do
4:    $j \leftarrow \arg \max_{j \in [n]} d(j, S)$ 
5:   (pick farthest point from  $S$ )
6:    $S \leftarrow S \cup \{j\}$ 
7: end
8: return  $S$ 
```

Remark

More concisely: start with any point; repeatedly add the point **farthest** from the current set.

Execution example ($k = 4$):



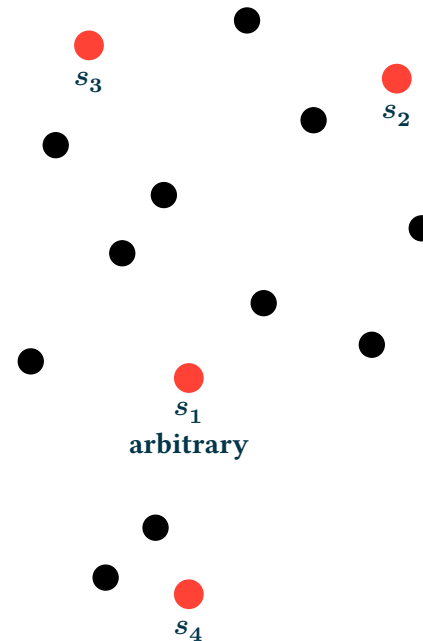
Algorithm 7: Greedy k -Centre

```
1: Input: points  $[n]$ , distances  $D$ , integer  $k$ 
2:  $S \leftarrow \{i\}$  for arbitrary  $i \in [n]$ 
3: while  $|S| < k$  do
4:    $j \leftarrow \arg \max_{j \in [n]} d(j, S)$ 
5:   (pick farthest point from  $S$ )
6:    $S \leftarrow S \cup \{j\}$ 
7: end
8: return  $S$ 
```

Remark

More concisely: start with any point; repeatedly add the point **farthest** from the current set.

Execution example ($k = 4$):



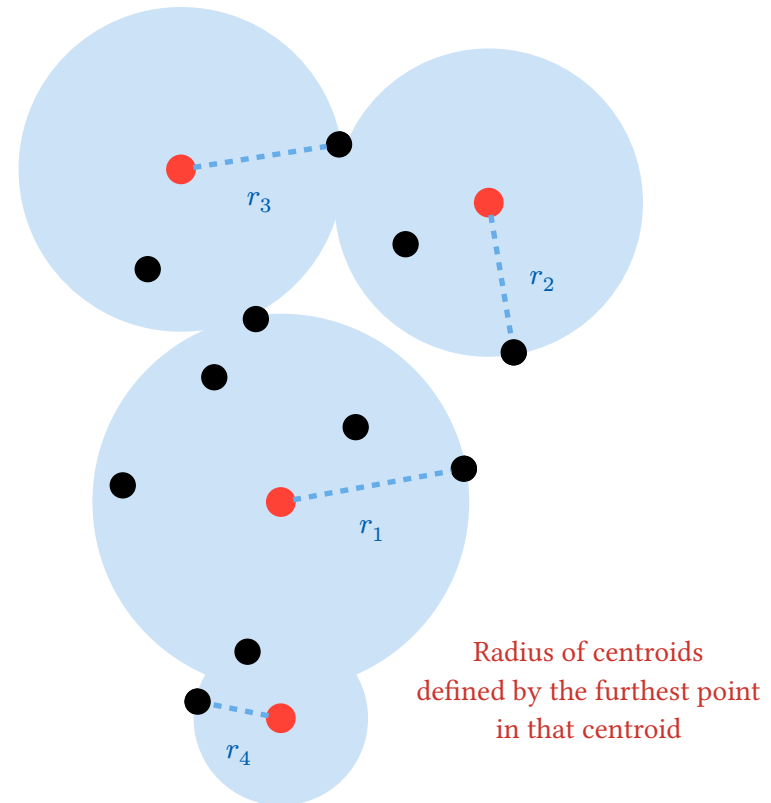
Algorithm 8: Greedy k -Centre

```
1: Input: points  $[n]$ , distances  $D$ , integer  $k$ 
2:  $S \leftarrow \{i\}$  for arbitrary  $i \in [n]$ 
3: while  $|S| < k$  do
4:    $j \leftarrow \arg \max_{j \in [n]} d(j, S)$ 
5:   (pick farthest point from  $S$ )
6:    $S \leftarrow S \cup \{j\}$ 
7: end
8: return  $S$ 
```

 Remark

More concisely: start with any point; repeatedly add the point **farthest** from the current set.

Execution example ($k = 4$):



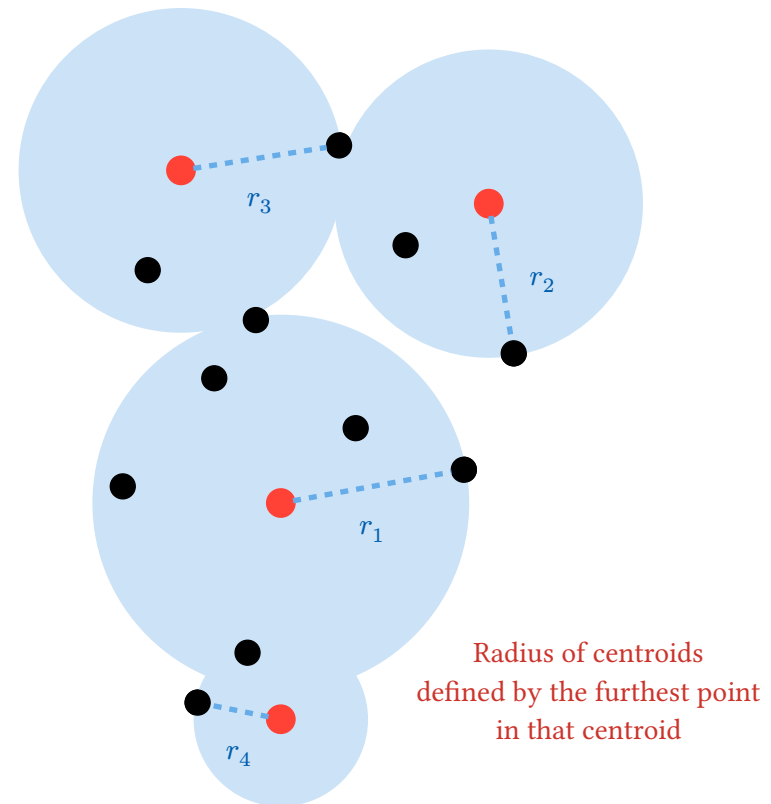
Algorithm 9: Greedy k -Centre

```
1: Input: points  $[n]$ , distances  $D$ , integer  $k$ 
2:  $S \leftarrow \{i\}$  for arbitrary  $i \in [n]$ 
3: while  $|S| < k$  do
4:    $j \leftarrow \arg \max_{j \in [n]} d(j, S)$ 
5:   (pick farthest point from  $S$ )
6:    $S \leftarrow S \cup \{j\}$ 
7: end
8: return  $S$ 
```

Remark

More concisely: start with any point; repeatedly add the point **farthest** from the current set.

Execution example ($k = 4$):



How Good is the greedy algorithm?

2. Approximation Algorithms

Definition 5 An algorithm A for a **minimisation** problem Π is an **f -approximation** for Π if

$$A(I) \leq f \cdot \text{OPT}(I)$$

for every instance I of Π . Here $A(I)$ is A 's output and $\text{OPT}(I)$ the optimal value.

Definition 7 An algorithm A for a **minimisation** problem Π is an **f -approximation** for Π if

$$A(I) \leq f \cdot \text{OPT}(I)$$

for every instance I of Π . Here $A(I)$ is A 's output and $\text{OPT}(I)$ the optimal value.

Definition 8 An algorithm A for a **maximisation** problem Π is an **f -approximation** for Π if

$$A(I) \geq f \cdot \text{OPT}(I)$$

for every instance I of Π . (Naturally $f \leq 1$.)

(Sometimes we say $f' \cdot A(I) \geq \text{OPT}(I)$ with $f' = 1/f \geq 1$)

Definition 9 An algorithm A for a **minimisation** problem Π is an **f -approximation** for Π if

$$A(I) \leq f \cdot \text{OPT}(I)$$

for every instance I of Π . Here $A(I)$ is A 's output and $\text{OPT}(I)$ the optimal value.

Definition 10 An algorithm A for a **maximisation** problem Π is an **f -approximation** for Π if

$$A(I) \geq f \cdot \text{OPT}(I)$$

for every instance I of Π . (Naturally $f \leq 1$.)

(Sometimes we say $f' \cdot A(I) \geq \text{OPT}(I)$ with $f' = 1/f \geq 1$)

Remark

We don't define $A(I) \leq \text{OPT}(I) + f$ because "hardly any problems will satisfy this."

Back to k -centre. It is a **minimisation** problem, so:

- I = an instance of k -centre
- $\text{OPT}(I) := \min_{S \subseteq [n], |S|=k} \max_{i \in [n]} d(i, S)$

Back to k -centre. It is a **minimisation** problem, so:

- I = an instance of k -centre
- $\text{OPT}(I) := \min_{S \subseteq [n], |S|=k} \max_{i \in [n]} d(i, S)$

Theorem 2 Proposition: The greedy algorithm is a **2-approximation** for the k -centre problem.

Back to k -centre. It is a **minimisation** problem, so:

- I = an instance of k -centre
- $\text{OPT}(I) := \min_{S \subseteq [n], |S|=k} \max_{i \in [n]} d(i, S)$

Theorem 3 Proposition: The greedy algorithm is a **2-approximation** for the k -centre problem.

That is: if S is the greedy output ($|S| = k$), then

$$\max_{i \in [n]} d(i, S) \leq 2 \cdot \text{OPT}(I).$$

Back to k -centre. It is a **minimisation** problem, so:

- I = an instance of k -centre
- $\text{OPT}(I) := \min_{S \subseteq [n], |S|=k} \max_{i \in [n]} d(i, S)$

Theorem 4 Proposition: The greedy algorithm is a **2-approximation** for the k -centre problem.

That is: if S is the greedy output ($|S| = k$), then

$$\max_{i \in [n]} d(i, S) \leq 2 \cdot \text{OPT}(I).$$

Two messages:

1. The greedy gives a solution at most **twice** the optimal radius.
2. **No polynomial-time algorithm can do better** (unless $P = NP$) — we will not prove this.

3. Proof of 2-Approximation

Let $S^* = \{j_1, j_2, \dots, j_k\} \subseteq [n]$ be an **optimal solution** and

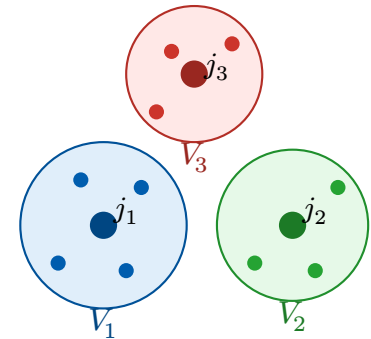
$$r^* := \max_{i \in [n]} d(i, S^*) \quad \text{-- the optimal radius of } S^*.$$

Let $S^* = \{j_1, j_2, \dots, j_k\} \subseteq [n]$ be an **optimal solution** and

$$r^* := \max_{i \in [n]} d(i, S^*) \quad \text{-- the optimal radius of } S^*.$$

S^* induces a **partition** $V_1 \cup V_2 \cup \dots \cup V_k = [n]$:

- Place $\ell \in [n]$ in V_i if among $\{j_1, \dots, j_k\}$ it is closest to j_i (break ties arbitrarily).

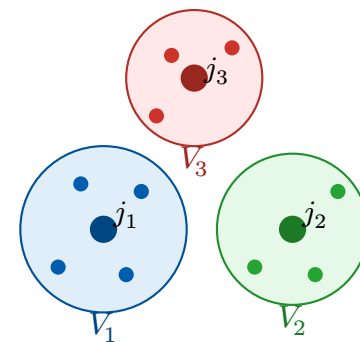


Let $S^* = \{j_1, j_2, \dots, j_k\} \subseteq [n]$ be an **optimal solution** and

$$r^* := \max_{i \in [n]} d(i, S^*) \quad \text{-- the optimal radius of } S^*.$$

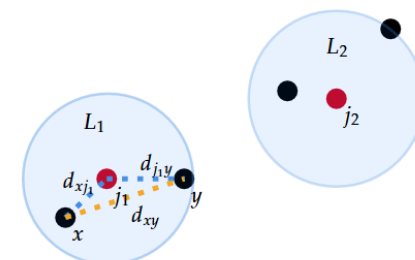
S^* induces a **partition** $V_1 \cup V_2 \cup \dots \cup V_k = [n]$:

- Place $\ell \in [n]$ in V_i if among $\{j_1, \dots, j_k\}$ it is closest to j_i (break ties arbitrarily).



Observation 3 Let $i \in [k]$ and $x, y \in V_i$. Then $d_{xy} \leq 2r^*$.

Proof: $d_{xy} \leq d_{xj_i} + d_{j_iy} \leq r^* + r^* = 2r^*$. $\square$



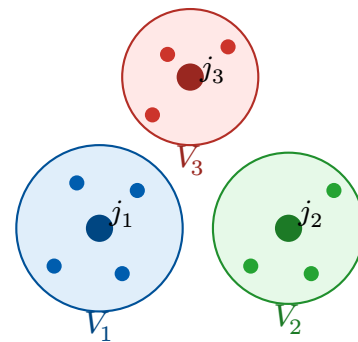
By triangle inequality
the blue path is at most
twice as long as the orange path

Let $S^* = \{j_1, j_2, \dots, j_k\} \subseteq [n]$ be an **optimal solution** and

$$r^* := \max_{i \in [n]} d(i, S^*) \quad \text{-- the optimal radius of } S^*.$$

S^* induces a **partition** $V_1 \cup V_2 \cup \dots \cup V_k = [n]$:

- Place $\ell \in [n]$ in V_i if among $\{j_1, \dots, j_k\}$ it is closest to j_i (break ties arbitrarily).

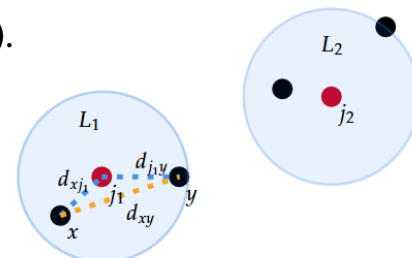


Observation 4 Let $i \in [k]$ and $x, y \in V_i$. Then $d_{xy} \leq 2r^*$.

Proof: $d_{xy} \leq d_{xj_i} + d_{j_iy} \leq r^* + r^* = 2r^*$. $\square$

Let $S_+ = \{s_1, \dots, s_k\}$ be the **greedy output**. Set $r := \max_{i \in [n]} d(i, S_+)$ (radius of S_+).

Current goal: $r \leq 2r^*$.



By triangle inequality
the blue path is at most
twice as long as the orange path

We consider two cases based on how the greedy centres S_+ interact with the optimal partition:

Case 1: $|S_+ \cap V_i| = 1$ for all $i \in [k]$.

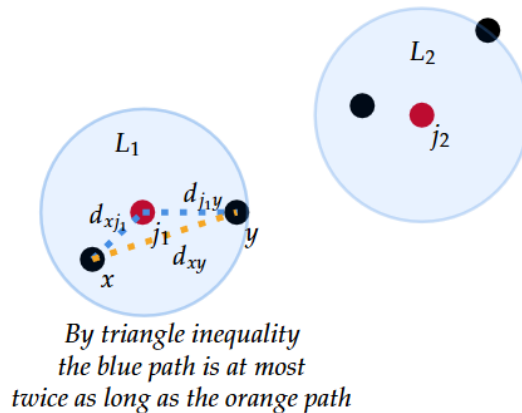
Each partition class contains **exactly one** greedy centre.

By the observation, for any $\ell \in [n]$ lying in V_i :

$$d(\ell, S_+) \leq d(\ell, s_i) \leq 2r^*$$

where $s_i \in S_+ \cap V_i$. (Since $\ell, s_i \in V_i$ implies $d(\ell, s_i) \leq 2r^*$.)

Done! $r \leq 2r^*$. $\square$



We consider two cases based on how the greedy centres S_+ interact with the optimal partition:

Case 1: $|S_+ \cap V_i| = 1$ for all $i \in [k]$.

Each partition class contains **exactly one** greedy centre.
By the observation, for any $\ell \in [n]$ lying in V_i :

$$d(\ell, S_+) \leq d(\ell, s_i) \leq 2r^*$$

where $s_i \in S_+ \cap V_i$. (Since $\ell, s_i \in V_i$ implies $d(\ell, s_i) \leq 2r^*$.)

Done! $r \leq 2r^*$. $\square$

Case 2: $\exists i \in [k]$ s.t. $|S_+ \cap V_i| \geq 2$.

Some V_i contains **two** greedy centres; call them s, s' (WLOG s was chosen first).

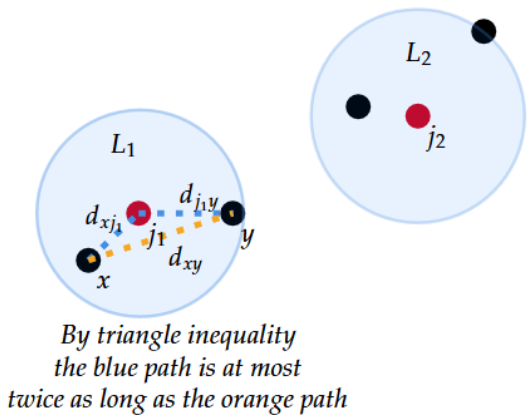
By the observation: $d_{ss'} \leq 2r^*$ (since $s, s' \in V_i$).

When s' was chosen, s was **already in S_+** . The greedy picks the **farthest** point from the current S_+ , so at that moment every other point p satisfies:

$$d(p, S_+) \leq d(s', S_+^*) \leq d_{ss'} \leq 2r^*,$$

where S_+^* is S_+ at the moment before s' was added to it.

Hence $r = \max_{i \in [n]} d(i, S_+) \leq 2r^*$. $\square$



4. The k -Suppliers Problem

Problem 11 The k -Suppliers Problem

Given: $[n] = A \cup B$, $k \in \mathbb{N}$, $|A| \geq k$.

- A = **suppliers** (facilities)
- B = **clients**

Find: $S \subseteq A$, $|S| = k$, minimising

$$\max_{b \in B} d(b, S)$$

Problem 12 The k -Suppliers Problem

Given: $[n] = A \cup B$, $k \in \mathbb{N}$, $|A| \geq k$.

- $A = \text{suppliers}$ (facilities)
- $B = \text{clients}$

Find: $S \subseteq A$, $|S| = k$, minimising

$$\max_{b \in B} d(b, S)$$

For each client $b \in B$, let $a_b \in A$ be the **nearest supplier** to b .

Problem 13 The k -Suppliers Problem

Given: $[n] = A \cup B$, $k \in \mathbb{N}$, $|A| \geq k$.

- A = **suppliers** (facilities)
- B = **clients**

Find: $S \subseteq A$, $|S| = k$, minimising

$$\max_{b \in B} d(b, S)$$

For each client $b \in B$, let $a_b \in A$ be the **nearest supplier** to b .

The Algorithm:

1. Apply the k -centre 2-approx to B . Let $S' \subseteq B$ be the output.
2. Return $S := \{a_b : b \in S'\}$. If $|S| < k$ add arbitrarily suppliers

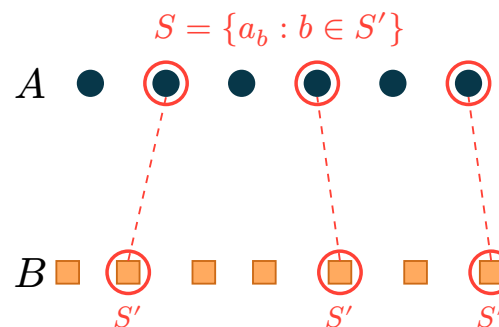
Problem 14 The k -Suppliers Problem

Given: $[n] = A \cup B$, $k \in \mathbb{N}$, $|A| \geq k$.

- A = **suppliers** (facilities)
- B = **clients**

Find: $S \subseteq A$, $|S| = k$, minimising

$$\max_{b \in B} d(b, S)$$



For each client $b \in B$, let $a_b \in A$ be the **nearest supplier** to b .

The Algorithm:

1. Apply the k -centre 2-approx to B . Let $S' \subseteq B$ be the output.
2. Return $S := \{a_b : b \in S'\}$.

If $|S| < k$ add arbitrarily suppliers

Theorem 5 Proposition: The above algorithm is a **3-approximation** for the k -suppliers problem.

Theorem 6 Proposition: The above algorithm is a **3-approximation** for the k -suppliers problem.

Proof. Let $r^* = \min_{S \subseteq A, |S|=k} \max_{b \in B} d(b, S)$. Fix any $b \in B$; let $b' \in S'$ be its nearest greedy centre.

Theorem 7 Proposition: The above algorithm is a **3-approximation** for the k -suppliers problem.

Proof. Let $r^* = \min_{S \subseteq A, |S|=k} \max_{b \in B} d(b, S)$. Fix any $b \in B$; let $b' \in S'$ be its nearest greedy centre.

Claim : For every $b \in B$: $d(b, S') \leq 2r^*$.

(proved on next slide)

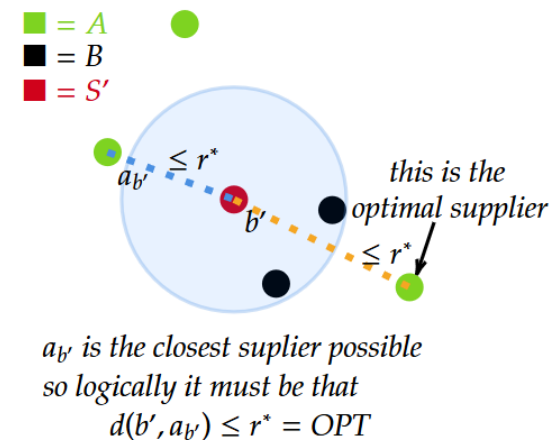
Theorem 8 Proposition: The above algorithm is a **3-approximation** for the k -suppliers problem.

Proof. Let $r^* = \min_{S \subseteq A, |S|=k} \max_{b \in B} d(b, S)$. Fix any $b \in B$; let $b' \in S'$ be its nearest greedy centre.

Claim : For every $b \in B$: $d(b, S') \leq 2r^*$.

(proved on next slide)

Since $a_{b'}$ is the nearest supplier to b' and OPT covers every client within r^* : $d(b', a_{b'}) \leq r^*$.



Theorem 9 Proposition: The above algorithm is a **3-approximation** for the k -suppliers problem.

Proof. Let $r^* = \min_{S \subseteq A, |S|=k} \max_{b \in B} d(b, S)$. Fix any $b \in B$; let $b' \in S'$ be its nearest greedy centre.

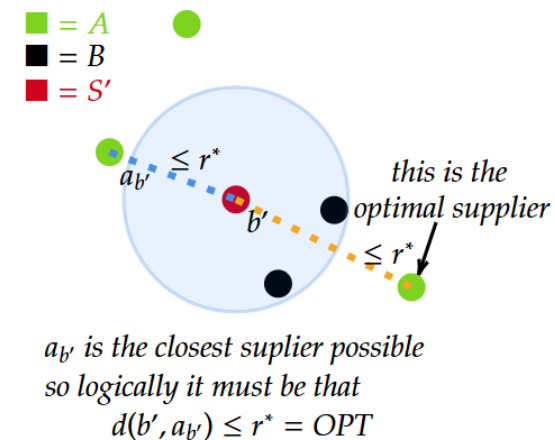
Claim : For every $b \in B$: $d(b, S') \leq 2r^*$.

(proved on next slide)

Since $a_{b'}$ is the nearest supplier to b' and OPT covers every client within r^* : $d(b', a_{b'}) \leq r^*$.

By the triangle inequality and the claim:

$$d(b, a_{b'}) \leq \underbrace{d(b, b')}_{\leq 2r^*} + \underbrace{d(b', a_{b'})}_{\leq r^*} \leq 3r^*. \quad \square$$



Theorem 10 Proposition: The above algorithm is a **3-approximation** for the k -suppliers problem.

Proof. Let $r^* = \min_{S \subseteq A, |S|=k} \max_{b \in B} d(b, S)$. Fix any $b \in B$; let $b' \in S'$ be its nearest greedy centre.

Claim : For every $b \in B$: $d(b, S') \leq 2r^*$.

(proved on next slide)

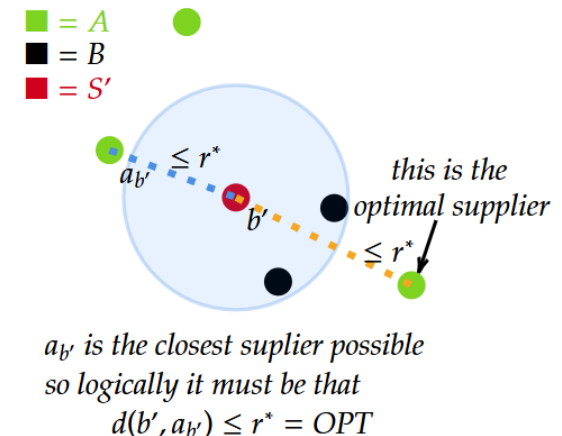
Since $a_{b'}$ is the nearest supplier to b' and OPT covers every client within r^* : $d(b', a_{b'}) \leq r^*$.

By the triangle inequality and the claim:

$$d(b, a_{b'}) \leq \underbrace{d(b, b')}_{\leq 2r^*} + \underbrace{d(b', a_{b'})}_{\leq r^*} \leq 3r^*. \quad \square$$

Remark

One can show that no polynomial-time f -approximation with $f < 3$ exists for k -suppliers (unless $P = NP$).



Claim : For every $b \in B$: $d(b, S') \leq 2r^*$.

Proof (by contradiction). Suppose $d(b, S') > 2r^*$ for some $b \in B$.

Claim : For every $b \in B$: $d(b, S') \leq 2r^*$.

Proof (by contradiction). Suppose $d(b, S') > 2r^*$ for some $b \in B$.

- $b \notin S'$ (else $d(b, S') = 0$).
- When each $b'_j \in S'$ was selected, greedy chose the **farthest** point, so: $d(b'_j, S'_{j-1}) \geq d(b, S'_{j-1}) \geq d(b, S') > 2r^*$.
- Hence **every pair** in $S' \cup \{b\}$ is $> 2r^*$ apart — a set of $k + 1$ points.

Claim : For every $b \in B$: $d(b, S') \leq 2r^*$.

Proof (by contradiction). Suppose $d(b, S') > 2r^*$ for some $b \in B$.

- $b \notin S'$ (else $d(b, S') = 0$).
- When each $b'_j \in S'$ was selected, greedy chose the **farthest** point, so: $d(b'_j, S'_{j-1}) \geq d(b, S'_{j-1}) \geq d(b, S') > 2r^*$.
- Hence **every pair** in $S' \cup \{b\}$ is $> 2r^*$ apart — a set of $k + 1$ points.

The optimal solution uses only k suppliers. By **pigeonhole**, two points $u, v \in S' \cup \{b\}$ share the same nearest supplier $a \in A$:

$$d(u, v) \leq d(u, a) + d(a, v) \leq r^* + r^* = 2r^*.$$

But $d(u, v) > 2r^*$ — **contradiction**. $\square$